

计算机图形学课程作业与大作业全流程配置指南

引言

本视频详细讲解了计算机图形学课程中**平时作业与大作业的完整开发环境配置流程、各项目核心要求及实现要点**。面向初学者，系统梳理了从 CMake 构建系统配置、Visual Studio/IDE 集成、OpenGL 项目编译运行，到六大核心作业（三角形绘制、五星红旗、变换动画、四元数相机、实例化渲染、光照与纹理）及两个 Bonus 实验（Alpha 测试/Blending、视锥体剔除）的技术路径。同时，深入解析了大作业（Final Project）的模块化工程搭建规范——涵盖源码结构设计、第三方库管理、CMakeLists.txt 编写逻辑与跨平台构建实践。全文以“可执行、可复现、可扩展”为原则，是图形学实践落地的关键操作手册。

背景

本课程基于 OpenGL 4.x 与现代 C++ (C++17) 技术栈，采用模块化教学框架：所有作业均构建于统一的 cg-project 基础代码库之上，该库封装了窗口管理 (GLFW)、OpenGL 函数加载 (glad)、数学运算 (glm)、GUI 控件 (imgui)、图像加载 (stb_image) 等核心依赖。课程强调**工程能力先行**——学生需首先掌握跨平台构建工具链 (CMake + IDE)，而非仅关注算法逻辑。值得注意的是，课程刻意规避了“一键式脚本”，要求学生手动理解 CMakeLists.txt 中的 add_executable、target_link_libraries、include_directories 等关键指令，从而建立对编译链接全过程的底层认知。这种设计源于图形学开发的现实复杂性：不同操作系统 (Windows/macOS/Linux) 的编译器 (MSVC/Clang/GCC)、SDK 版本、GPU 驱动差异，均可能导致相同代码构建失败或运行异常，因此环境配置本身即为第一道重要考核。

主要观点

一、CMake 构建系统配置：从 GUI 到命令行的双重实践

视频开篇即强调 CMake 是本课程的基石工具。学生需安装 **CMake ≥ 3.15** (推荐 3.23+)，并配合本地编译器使用：Windows 推荐 Visual Studio 2019/2022 (勾选“使用 C++ 的桌面开发”及“Windows 10/11 SDK”)；macOS 使用 Xcode；Linux 使用 GCC。配置分为两种方式：

- **GUI 方式**：启动 cmake-gui → 设置源码路径 (含顶层 CMakeLists.txt) 与构建路径 (如 build/) → 点击 *Configure* (自动检测默认编译器) → 选择生成器 (如 Visual Studio 17 2022) → 点击 *Generate* → 生成 .sln 工程文件。

- **命令行方式**：在终端中执行 `cmake -B build -S .`（-B 指定构建目录，-S 指定源码目录），随后进入 `build/` 目录运行 `cmake --build .` 即可调用 `cl.exe` 或 `g++` 编译。

该流程本质是将 `CMakeLists.txt` 中声明的构建规则（如 `add_executable(getstart src/getstart.cpp)`）翻译为 IDE 或 Makefile 可识别的指令。`CMakeLists.txt` 不是黑盒，而是项目架构的“宪法”——它定义了源码组织、依赖链接与输出路径，是后续所有作业可维护性的根本保障。

视频截图生成失败，请重试或手动插入截图

视频截图生成失败，请重试或手动插入截图

二、六大核心作业：从 OpenGL 基础到高级渲染管线

课程作业按技术难度递进设计，覆盖图形学核心概念：

- **GetStart（三角形绘制）**：学习顶点缓冲对象（VBO）、顶点数组对象（VAO）、着色器编译链接（GLSL）、`glDrawArrays` 渲染流程，是后续所有作业的“Hello World”。
 - **五星红旗**：巩固顶点数据构造（五角星可拆解为10个三角形）、`glClearColor` 设置旗面背景色、坐标系与比例控制（窗口设为3:2）。
 - **变换动画（Stanford Bunny）**：通过 `glm::mat4` 构造平移/旋转/缩放矩阵，理解 MVP 变换中 Model 矩阵的动态更新，实现周期性运动。
 - **四元数相机（Rabbit Scene）**：实现 Quaternion 类，用 `glm::quat` 替代欧拉角避免万向节锁；支持 WASD 移动、鼠标俯仰/偏航、空格键切换正交/透视投影，并理解近裁剪面（near plane）对物体裁切的影响。
 - **实例化渲染（Asteroid Belt）**：用 `glDrawArraysInstanced` 替代循环调用 `glDrawArrays`，将1万个陨石的变换矩阵存入 SSBO 或 UBO，显著提升帧率（尤其在非60Hz显示器上）。
 - **光照与纹理（Phong/Blinn-Phong）**：实现环境光（Ambient）、漫反射（Lambert）、高光（Phong）三部分；通过 `texture()` 采样贴图，结合 `mix()` 实现纹理混合（Blend Texture）；用程序化着色器（Procedural Texture）生成棋盘格，理解 `fract()` 与 `step()` 的数学本质。
- 每项作业均提供 `demo/` 下的参考效果，鼓励学生对比“修改前 vs 修改后”，在视觉反馈中验证代码正确性。

视频截图生成失败，请重试或手动插入截图

视频截图生成失败，请重试或手动插入截图

三、Bonus 实验：深入透明渲染与空间优化技术

两个 Bonus 作业拓展了课程深度：

- **Bonus1 (Alpha Testing & Blending)** : 对比 discard (粗暴剔除像素) 与 `glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)` (加权混合) 的差异; 指出传统 Alpha Blending 的深度排序缺陷, 并引入 **Depth Peeling** 算法——通过多 Pass 渲染 (如10次) 逐层提取前景/背景颜色, 利用 `glFramebufferTexture2D` 将深度/颜色附件绑定至 FBO, 最终合成正确透明效果。此过程涉及 Framebuffer Object (FBO)、Renderbuffer、Query Objects 等高级 OpenGL 对象。
- **Bonus2 (Frustum Culling)** : 为每个陨石构建轴对齐包围盒 (AABB), 将其从模型空间经 MVP 矩阵变换至裁剪空间, 再与标准视锥体 (6个平面方程) 做相交测试; 强调 **CPU端串行剔除** (简单可靠) 与 **GPU端并行剔除** (使用 Transform Feedback 将计算卸载至 GPU) 的性能权衡; 指出 AABB 在世界空间的旋转表现是其变换后的自然结果, 无需额外处理。
这些实验直指工业级引擎优化痛点, 如《Unity》的 Occlusion Culling、《Unreal》的 Hierarchical Z-Buffering, 让学生理解“看不见的物体不渲染”这一黄金法则的工程实现。

视频截图生成失败, 请重试或手动插入截图

视频截图生成失败, 请重试或手动插入截图

四、大作业 (Final Project) : 模块化工程架构实战

大作业要求学生从零搭建一个可扩展的图形学应用, 其工程结构必须严格遵循:

- **source/**: 存放全部自研源码 (base/ 提供基础框架, project/ 存放主逻辑);
- **media/**: 集中管理模型 (.obj)、贴图 (.png)、配置文件等资源;
- **external/**: 存放第三方库 (glad、glm、glfw、imgui、stb_image), **显式移除 tinyobjloader** (因作业要求自研 OBJ 解析器);
- **CMakeLists.txt**: 核心配置文件, 需包含:
`cmake_minimum_required(VERSION 3.15)project(FinalProject LANGUAGES CXX)`

复制 media/ 到 build/ 目录

```
file(COPY ${CMAKE_SOURCE_DIR}/media DESTINATION ${CMAKE_BINARY_DIR})
```

添加 external/ 下各子库

```
add_subdirectory(external/glad)add_subdirectory(external/glm)
```

定义源文件列表

```
file(GLOB_RECURSE SOURCES "source/*.cpp" "source/*.h")
```

创建可执行文件并链接依赖

```
add_executable(${PROJECT_NAME} ${SOURCES})target_link_libraries(${PROJECT_NAME}
glad glfw glm imgui stb_image)
```

视频截图生成失败，请重试或手动插入截图

视频截图生成失败，请重试或手动插入截图

关键挑战在于：base/model.h 原依赖 tinyobjloader，需重写 OBJLoader 类，解析 .obj 文件的 v（顶点）、vt（纹理坐标）、vn（法线）、f（面片）字段，并支持导出功能。此过程强制学生深入理解网格数据内存布局与 OpenGL 顶点属性绑定机制。

结论

本课程的终极目标并非教会学生“如何画一个三角形”，而是培养其作为图形学工程师的**系统性工程能力**：从构建工具链（CMake）的精确配置，到 OpenGL API 的严谨调用；从数学原理（四元数、MVP 矩阵、Phong 光照模型）的代码实现，到性能优化（实例化、视锥剔除、Depth Peeling）的工程取舍；最终落脚于一个结构清晰、可维护、可扩展的 Final Project 工程。其中，**CMakeLists.txt 是项目的灵魂**——它将分散的源码、资源、库整合为统一构建单元；**手动编写 OBJ 加载器是能力的分水岭**——它迫使学生穿透抽象层，直面数据格式与内存管理的本质。建议学生：

1. **严格遵循视频演示的目录结构与 CMake 指令**，避免跳过“看似繁琐”的配置步骤；
2. **每个作业务必先运行 demo/ 效果，再动手修改**，以视觉反馈驱动调试；
3. **大作业阶段，优先实现 OBJLoader 与基础渲染管线**，再逐步叠加材质、光照、动画等模块，避免过度设计；
4. **Bonus 实验虽非强制，但 Depth Peeling 与 Frustum Culling 是面试高频考点**，建议投入时间攻克。

图形学的魅力正在于：每一行代码的改动，都在屏幕上实时生成一个可交互的三维世界——而这份从零构建世界的掌控感，正是本课程赋予每位学习者最珍贵的礼物。